Supplementary Material for "Trajecta: A Heterogeneous, Traffic-Aware and Infrastructure-Conscious Smart Logistics Optimization Framework"

This file holds material cut from the main paper. All values come from the experiments reported in the main paper. Repository: https://github.com/tmdinesh/Trajecta

S1. Vehicle Specification and Cost Rates

Table S1. Fleet specification

Vehicle	Capacity (kg / m^3)	Speed (km/h)	Fixed cost (Rs.)
2W-Petrol	35 / 0.18	34	0
2W-EV	30 / 0.16	32	0
4W-Petrol	350 / 2.5	42	70
4W-Diesel	700 / 5.5	40	100
4W-EV	400 / 3.0	38	65
Truck-Diesel	4000 / 25	35	180
Truck-EV	2500 / 18	33	160

Table S2. Cost rates, traffic multipliers and service-time parameters

Parameter	Value
Energy rate (Rs./km)	Petrol 4.5; diesel 7; electric 1.1
Wages / maintenance (Rs./km)	2 / 0.55
Late penalty	Rs. 100 per day
Packaging	Rs. 35 per flagged order
Toll	Rs. 30 per route
EV charging	Rs. 0.35/km
Cost scale factor	1.16
Traffic multipliers	Morning 1.30; evening 1.25; midday 1.05; night 0.90
Weather and lane factors	1.0 and 1.0
Service time	Base 8 min; 0.05 min/kg; 5 min/m^3; +4 min fragile; +6 min cooling; noise SD 2 min; $\gamma = \delta = 1.0$

S2. Synthetic Data Generation and Public Instances

- **Orders:** 30 synthetic orders over 18 named city entries around Coimbatore (including the depot); file synthetic_30_orders.csv.
- **Ranges:** weight 4–55 kg; volume 0.015–0.24 m^3; due-time offset 4–8 h; arrival time 6–10 h; 15% fragile; 20% cooling.
- **Distances:** haversine distance × road factor 1.18. Coordinates and infrastructure constraints are generated by the implementation.
- **Infrastructure rules:** synthetic assumptions based on city-pair rules; they are not real infrastructure data.
- **3D capacity check:** greedy placement of boxes into the available free spaces, trying axis permutations.
- **Public instances:** Solomon c101, r101, rc101 and Cordeau MDVRP p01, p02. Provenance is recorded in PROVENANCE.txt in the repository.

S3. Formal Formulation (Full)

Sets and data. O: orders ($|O| = 30$); V: vehicle types (7); C: cities including the depot 0 ($|C| = 18$); $c(o) \in C \setminus \{0\}$: destination city of order o. Order data: weight w_o, volume V_o, due time D_o, fragile flag f_o, cooling flag c_o. Vehicle data: weight capacity W_v, volume capacity Q_v, speed s_v, fixed cost (Table S1).

Decision variables. $x_{ov} \in \{0,1\}$: order o is carried by vehicle v (batch-to-vehicle assignment); π_v: visit order of the destination cities of $B_v = \{o : x_{ov} = 1\}$, starting and ending at the depot.

$$\min \ \Sigma_{v \in V} \ \text{TotalCost}(\pi_v, B(v)) \tag{S1}$$

subject to:

- Assignment: each order is carried by exactly one vehicle, $\Sigma_v x_{ov} = 1$ for all o.
- Weight capacity: $\Sigma_{o \in B(v)} w_o \le W_v$. Volume capacity: $\Sigma_{o \in B(v)} V_o \le Q_v$.
- 3D placement feasibility: Pack3D(B(v), v) = feasible, by the greedy free-space placement in S2.
- Infrastructure feasibility: Infra(π_v, v) = feasible for every segment of the route, under the synthetic city-pair rules in S2.
- Due times by penalty: arrival times follow from the travel-time and service-time models (S4). Lateness $L_o \ge 0$ (in days) enters the late-penalty term $PC = 100 \, \Sigma_o L_o$. Due times are soft.

S4. Equations (1)–(3): Parameter Definitions and Values Used

$$T_{ij}^{v,t} = (d_{ij} / s_v) \times \lambda_v \times \omega \times \tau_t \quad (1)$$

$$S_i = t_{base} + \alpha\, w_i + \beta\, V_i + \gamma(4 f_i + 6 c_i) + \delta\, \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2) \quad (2)$$

$$\text{TotalCost(route)} = EC + WF + PC + PK + MC + TC + CC \quad (3)$$

Table S3. Symbols in Eqs. (1)–(3) and the values used

Symbol	Meaning	Value used
d_{ij}	Segment distance	Haversine × 1.18
s_v	Vehicle base speed	32–42 km/h by type (Table S1)
λ_v, ω	Lane and weather multipliers	1.0, 1.0
τ_t	Time-of-day multiplier	1.30 / 1.25 / 1.05 / 0.90
t_{base}	Base service time	8 min
α, β	Service time per kg, per m^3	0.05 min/kg, 5 min/m^3
γ, δ	Weights on flag term and noise term	1.0, 1.0
f_i, c_i	Fragile and cooling flags (+4 and +6 min)	15% and 20% of orders
ε_i, σ	Gaussian noise and its SD	σ = 2 min
EC	Energy cost	4.5 / 7 / 1.1 Rs./km (petrol / diesel / electric)
WF	Wages plus fixed cost	2 Rs./km plus fixed cost (Table S1)
PC	Late penalty	Rs. 100 per late day
PK	Packaging	Rs. 35 per flagged order
MC	Maintenance	Rs. 0.55/km
TC	Tolls	Rs. 30 per route
CC	EV charging	Rs. 0.35/km

There is no separate worker-skill term in Eq. (2): the stochastic component is the Gaussian noise term, with no separate skill mean.

S5. Candidate Files

The repository holds all 56 candidates per seed in **candidate_routes.csv** and the per-seed selections (minimum cost, minimum distance, minimum penalty) in **selected_candidate_routes.csv**. Columns: seed, split, split_strategy, heuristic, cost_rs, distance_km, penalty_rs, ontime_pct, vehicles. The selections are made independently by argmin over the 56 candidates of each seed, with no scalarization.

S6. OR-Tools CP-SAT and Metaheuristic Comparison

- **CP-SAT:** time limit 10 s on the synthetic runs and 2 s on the benchmark runs. Synthetic status: 9 FEASIBLE and 1 UNKNOWN out of 10 seeds. Average optimality gap 90.31%, so no optimality conclusions are drawn. Statistics in Table 3 of the main paper use the 9 feasible runs.
- **Same cost:** CP-SAT, ALNS, GA and Tabu Search are all scored with the seven-factor cost.
- **ALNS, GA, Tabu Search:** Trajecta's best-candidate means are lower in cost and distance. ALNS and Tabu Search have a lower mean penalty (Rs. 23.20) and slightly higher on-time delivery (99.33%) than Trajecta's minimum-cost route (Rs. 34.80; 99.00%).
- **Runtime:** Trajecta's runtime was not recorded in the result CSVs, so no runtime comparison is made.

S7. Full Ablation and Scalability Results

Table S4. Ablation, mean over 3 seeds per variant (baseline includes the DBSCAN split)

Variant	Mean cost (Rs.)	Mean distance (km)	Mean on-time (%)
Baseline	3,568.80	363.08	100.00
No 3D packing	3,568.80	363.08	100.00
No infrastructure	3,550.94	363.08	100.00
No DBSCAN	4,362.75	474.12	97.78
No penalty-risk split	3,572.84	363.87	100.00
No traffic	3,547.46	358.67	100.00
No stochastic service time	3,558.40	358.67	100.00

The baseline (Rs. 3,568.80) differs from the 10-seed mean in Table 3 (Rs. 4,278.20) because it uses a different seed set. Only 3 seeds per variant; no significance test.

Table S5. Scalability (2 seeds per size; OR-Tools CP-SAT)

Orders	OR-Tools cost / distance	Mean runtime (s)	Status
20	Rs. 1,994.16 / 212.18 km	≈ 10.03	Scored
30	Rs. 4,548.46 ± 9.59 / 492.09 ± 6.41 km	≈ 10.07	Scored
50	No feasible score	≈ 10.11	Saved runs infeasible
80	No feasible score	≈ 11.32	Saved runs infeasible; metaheuristic scores recorded as infeasible

No Trajecta numbers are reported for these sizes. Scaling beyond 30 orders was not successfully demonstrated for the baselines. Full results: scalability_summary.csv.

S8. Per-Instance Benchmark Results

Public benchmarks use a vehicle-count-first, distance-second objective, not the seven-factor cost. Values are means over 3 seeds (11, 22, 33), 2 s time limit; all methods were feasible in 15 of 15 runs.

Table S6. Mean vehicles and distance per instance

Instance	Trajecta	OR-Tools	Winner
Cordeau p01	12, 613.728	12, 609.241	OR-Tools
Cordeau p02	7, 517.315	7, 507.011	OR-Tools
Solomon c101	10, 828.937	10, 828.937	Tie
Solomon r101	23, 1,994.482	22, 1,789.111	OR-Tools
Solomon rc101	17, 1,976.042	18, 1,806.160	Trajecta

Trajecta wins 1, ties 1, loses 3. Total vehicle counts tie across the set; OR-Tools has the lower aggregate distance. Files: benchmark_algorithm_runs.csv, benchmark_algorithm_summary.csv.

S9. Reproducibility

Code, data and results: https://github.com/tmdinesh/Trajecta. Environment: Python 3.12.9, OR-Tools 9.15.6755, NumPy 2.3.5, pandas 3.0.1, SciPy 1.18.1, Matplotlib 3.11.2; AMD Ryzen 5 5600H, 15.4 GB RAM. Synthetic seeds 0–9; benchmark seeds 11, 22, 33; time limits 10 s (synthetic) and 2 s (benchmarks). Result files in the repository:

- candidate_routes.csv
- ortools_baseline.csv
- metaheuristic_results.csv
- ablation_summary.csv
- scalability_summary.csv
- benchmark_algorithm_runs.csv
- benchmark_algorithm_summary.csv
- assumptions.json
- PROVENANCE.txt
- figures/
- tables/

Exact command to rerun the experiment: *powershell -ExecutionPolicy Bypass -File .\run_pipeline.ps1*